Robotics Programming Challenge 2

Linear Jacobian, Jacobian derivative and trajectory planning of the Franka Emika Panda · MCTR911, Winter 2026 · **Team 6: The High Reach**

Name / ID: _____

This report documents the Jacobian and its derivative, the trajectory planner, the singularity analysis for the assigned scenario and the POSITION / VELOCITY experiments. The code is in *panda_kinematics/trajectory_node.py* (the instructor's template; the EDIT section holds the Challenge 1 code and the three new functions). Verification is done by *tools/check_challenge2.py* (22 checks, all pass) and the recorded runs are in *challenge2_results/*. The robot model is panda.xml from MuJoCo Menagerie, the same as in Challenge 1; only the seven arm joints are used (the gripper command is fixed).

1. Linear geometric Jacobian

Model. The DH table, the fixed link7→hand transform and the joint limits are those of Challenge 1 (standard DH, $A_i = \mathrm{Rot}_z(q_i)\,\mathrm{Trans}_z(d_i)\,\mathrm{Trans}_x(a_i)\,\mathrm{Rot}_x(\alpha_i)$):

i	α_i [rad]	a_i [m]	d_i [m]	q_{min} .. q_{max} [rad]
1	-1.5708	+0.0000	0.333	-2.8973 .. +2.8973
2	+1.5708	+0.0000	0.000	-1.7628 .. +1.7628
3	+1.5708	+0.0825	0.316	-2.8973 .. +2.8973
4	-1.5708	-0.0825	0.000	-3.0718 .. -0.0698
5	+1.5708	+0.0000	0.384	-2.8973 .. +2.8973
6	+1.5708	+0.0880	0.000	-0.0175 .. +3.7525
7	+0.0000	+0.0000	0.000	-2.8973 .. +2.8973

Derivation. A point p fixed to the hand, on a rigid body that rotates about an axis with unit direction z through the point o, moves with velocity $z \times (p - o)$. In standard DH, joint i rotates about z_{i-1} through the origin o_{i-1} of frame i−1, and the velocity of the hand origin is the sum over the joints, so $v = J(q)\,dq$ with the column of joint i

```
J_v,i = z_(i-1) x ( p_hand - o_(i-1) ),     i = 1..7
z_(i-1) = 3rd column, o_(i-1) = 4th column of T_0^(i-1) = A1 A2 ... A(i-1)
p_hand  = 4th column of T_0^hand
```

The result is a **3 x 7** matrix: rows are the hand velocity along the base x, y, z; column i is the hand velocity produced by 1 rad/s of joint i with the other joints frozen. Seven columns and three rows give a redundancy of four degrees of freedom (null space).

Worked columns at the end pose q = [+0.300, -0.100, +0.000, -0.800, +0.000, +1.600, +0.785]: p_hand = [+0.357, +0.110, +1.005].

Joint	z_{i-1}	o_{i-1}	Column $J_{v,i} = z \times (p - o)$
1	[+0.000, +0.000, +1.000]	[+0.000, +0.000, +0.000]	$(-p_y, p_x, 0)$ = [-0.110, +0.357, +0.000]
2	[-0.296, +0.955, +0.000]	[+0.000, +0.000, +0.333]	[+0.642, +0.199, -0.373]
4	[+0.296, -0.955, +0.000]	[+0.048, +0.015, +0.656]	[-0.334, -0.103, +0.323]
7	[+0.748, +0.231, -0.622]	[+0.277, +0.086, +1.071]	[-0.000, +0.000, -0.000] (axis passes through the hand origin)

Implementation. compute_jacobian(q) chains the DH matrices once (the same frames as the forward kinematics) and evaluates the cross product for each joint.

```
frames = _frames(q);  p_hand = frames[-1][:3, 3]
for i in range(7):  J[:, i] = np.cross(frames[i][:3, 2], p_hand - frames[i][:3, 3])
```

2. Jacobian derivative

The Jacobian depends on q, so it changes while the arm moves. Its time derivative is what turns joint accelerations into hand acceleration:

```
v_hand = J dq          a_hand = J ddq + (dJ/dt) dq
```

The second term is the acceleration the hand has even when the joint accelerations are zero (for one joint at constant speed it is the centripetal term w^2 r). dJ/dt is computed analytically by differentiating each column $z_i \times (p - o_i)$ with the product rule:

```
d col_i/dt = ( dz_i x (p - o_i) ) + ( z_i x (dp - do_i) )
w_i = sum_{k<i} z_k dq_k;   dz_i = w_i x z_i;   do_i = sum_{k<i} z_k x (o_i - o_k) dq_k;   dp = J dq
```

Each axis and origin moves only because the joints *before* it rotate, which gives the sums above. A central finite-difference version, (J(q + h dq) − J(q − h dq)) / 2h, is kept only as a check.

Check (tools/check_challenge2.py, 200 random poses in the joint limits)	Result
J vs MuJoCo's own Jacobian (mj_jac at the hand origin)	max difference 1.3e-15
J vs finite differences of the forward kinematics	max difference 1.8e-10
dJ/dt analytic vs finite differences	max difference 1.9e-10

3. Trajectory planning

3.1 Scenario: Team 6, "The High Reach"

	q (rad)	Hand position (m)	Distance from shoulder / max reach
Start (low)	[+0.000, +0.350, +0.000, -2.200, +0.000, +2.550, +0.785]	[+0.556, -0.000, +0.222]	66 %
End (high reach)	[+0.300, -0.100, +0.000, -0.800, +0.000, +1.600, +0.785]	[+0.357, +0.110, +1.005]	90 %

The arm starts low (hand 0.22 m above the floor, pointing down) and reaches up and out to 1.00 m, close to the top of the workspace (the hand origin reaches at most about 1.19 m). The elbow stays bent (-46°), so the arm is not fully stretched. Both poses are inside the joint limits (closest margin 41.8°) and MuJoCo reports no contacts along the path.

3.2 Method: quintic polynomial in joint space

```
q(t)   = q_start + s(tau) (q_end - q_start),          tau = t / T
s(tau) = 10 tau^3 - 15 tau^4 + 6 tau^5
dq(t)  = (30 tau^2 - 60 tau^3 + 30 tau^4)/T * (q_end - q_start)
ddq(t) = (60 tau - 180 tau^2 + 120 tau^3)/T^2 * (q_end - q_start)
```

The six coefficients of a fifth-order polynomial are fixed by six conditions: position, velocity and acceleration at both ends (s = 0, 1; s' = 0, 0; s'' = 0, 0). All joints share the same s(tau), so they start and stop together and the path in joint space is a straight line.

Moment	Position	Velocity	Acceleration
t = 0	q_start	0	0
t = T	q_end	0	0

Moment	Position	Velocity	Acceleration
t = T/2	midpoint	maximum, 15/8 · |dq|/T	0
t = 0.21 T, 0.79 T			maximum, $10/\sqrt{3} \cdot \|dq\|/T^2$

For joint 4, which moves most (|dq| = 1.40 rad) with T = 4 s: peak speed 0.656 rad/s (30 % of 2.175 rad/s) and peak acceleration 0.505 rad/s^2 (4 % of 12.5). A cubic would leave a non-zero acceleration at both ends and linear interpolation has an infinite one, so the quintic was chosen.

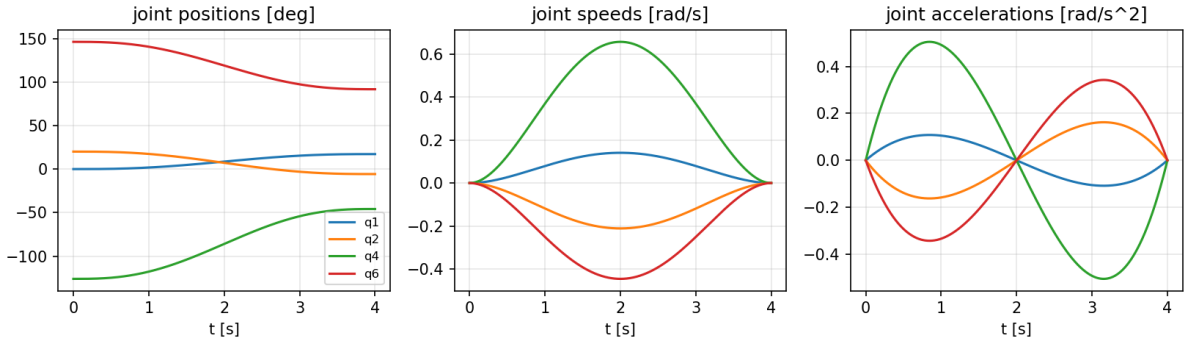


Figure 1: planned motion (positions, speeds, accelerations) of the joints that move. Every curve starts and ends at zero speed and acceleration.

plan_trajectory(q_start, q_end, T, dt) returns four arrays of equal length N = T/dt + 1 (times, positions, velocities, accelerations) and contains three safeguards:

- **Joint limits.** The quintic is monotonic, so every joint stays between its start and end value; checking the two end points guarantees the whole path (a goal outside the limits raises an error).
- **Speed and acceleration limits.** Peaks are known in closed form, so T must be at least $(15/8)|dq|/(0.6\ v_{max})$ and $\mathrm{sqrt}((10/\sqrt{3})|dq|/(0.6\ a_{max}))$ for every joint (60 % of Franka's published limits). A shorter T is stretched automatically. Here the minimum is 2.01 s and T = 4 s is used.
- **Singularity guard** (section 4).

4. Singularities near the top of the workspace

At a singularity J loses rank: the hand cannot move in some direction, and the joint speeds needed for a hand velocity v, $dq = J^{+}v$, grow like $1/\sigma$ with the small singular value σ. Close to the upper boundary of the workspace the arm is almost stretched, so this matters for Team 6. Two indicators of the 3 x 7 Jacobian are monitored along the path:

Indicator	Meaning	Threshold	Our path (min)	Arm straight up
$\sigma_{min}(J)$	hand speed per rad/s of joint speed in the worst direction	≥ 0.08	0.150	0.099
$w = \mathrm{sqrt}(\det(J\ J^{T}))$	manipulability: volume of the available hand velocities	≥ 0.03	0.079	0.0116

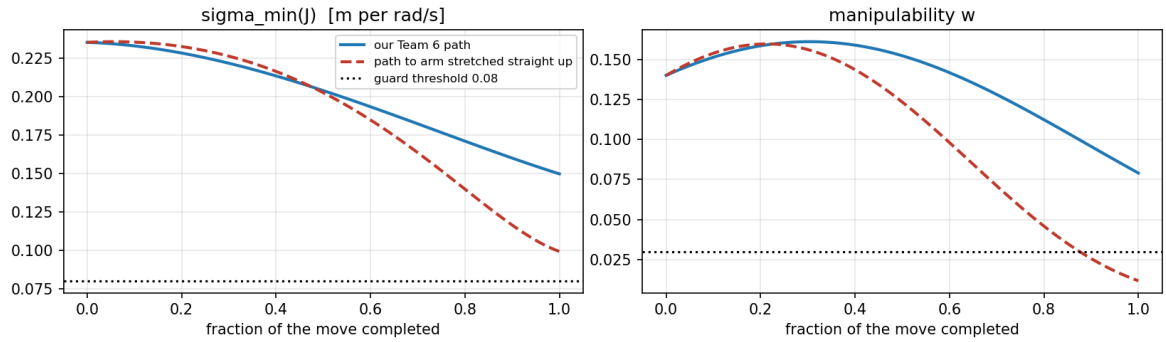


Figure 2: σ_{min} and w along our path (solid) and along a path ending with the arm stretched straight up (dashed red).

How the motion is kept safe. (1) The goal is at 90 % of full reach with a bent elbow, away from the stretched configuration (w = 0.079, σ_{min} = 0.150). (2) Planning is done in joint space, so J is never inverted and the joint speeds are bounded by 15/8 · |dq|/T whatever the pose (30 % of the speed limit here). (3) T is stretched if a limit would be exceeded. (4) The guard samples the path and, if σ_{min} or w falls below its threshold, backs the goal off along the same joint line by bisection and logs a warning; for a goal with the arm straight up it shortened the move to 88 %. **Limits:** only the position Jacobian is monitored, and a joint-space plan protects the joint speeds but does not keep the hand path straight; a Cartesian planner would need a damped pseudo-inverse $dq = J^T(J\,J^T + \lambda^2 I)^{-1}v$ and slower motion near singularities.

5. POSITION and VELOCITY control modes

Bridge behaviour (mujoco_ros.cpp): in POSITION mode the received numbers are copied to the actuator targets (ctrl = command). In VELOCITY mode the bridge adds command / simulation_frequency to the target **once for every message** (simulation_frequency = 1000). Each joint is a position servo, force = $k_p(\text{ctrl} - q) - k_v\,dq$ with k_v/k_p = 0.1 s for all joints. The node publishes q_traj[k] (POSITION) or dq_traj[k] (VELOCITY) by switching the two commented places in tick(); the bridge mode was changed to match. All runs used the real C++ bridge drawing at 20 Hz, with a logger recording planned and actual joint angles.

Set	Node	Msgs / s	Arm starts at	Purpose
A	trajectory_node.py as submitted	1000	all zeros	following the challenge text
B	same with the template's TRAJ_DT = 0.05	20	all zeros	the template's own rate
C	extended runner (set-up move first)	1000	Q_START	clean tracking comparison

5.1 Template node (sets A and B)

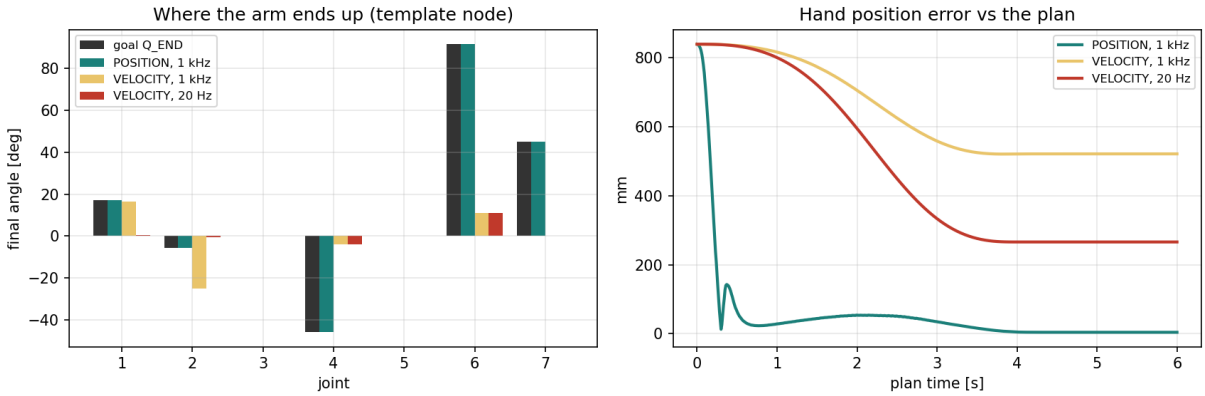


Figure 3: left: final joint angles against the goal; right: hand error against the plan (template node).

Run	Hand error RMS / max	Settled hand error	Commanded motion delivered
POSITION, 1 kHz	165 / 838 mm	4.3 mm	n/a (absolute target)
POSITION, 20 Hz	178 / 838 mm	4.3 mm	n/a (absolute target)
VELOCITY, 1 kHz	700 / 838 mm	521 mm	joints 1, 2: 96 %, 98 %; joints 4, 6: about 0
VELOCITY, 20 Hz	610 / 838 mm	266 mm	2.0 % (= 1/50)

- The simulator starts with all joints at 0, but the template does not move the arm to Q_START first. In **POSITION** mode the arm snaps to Q_START in about 0.7 s (the large first-second error) and then tracks; it settles within 0.20°.
- In **VELOCITY** mode the bridge's target starts at 0 and only accumulates, so the arm performs the relative motion Q_END − Q_START from zeros and never reaches Q_END. Joints 4 and 6 do not move at all: their integrated targets (+1.4 rad, −0.95 rad) lie outside the actuator range (joint 4 < −0.07, joint 6 > −0.0175) and are clamped.
- At the template's TRAJ_DT = 0.05 only 2.0 % of the commanded motion arrives (20 messages per second, each adding command/1000). **TRAJ_DT was therefore set to 0.001 (1 kHz)**, the only change below the template's marked line besides Q_START / Q_END; setting 0.05 again reproduces the template behaviour.

5.2 Clean comparison from the same start pose (set C)

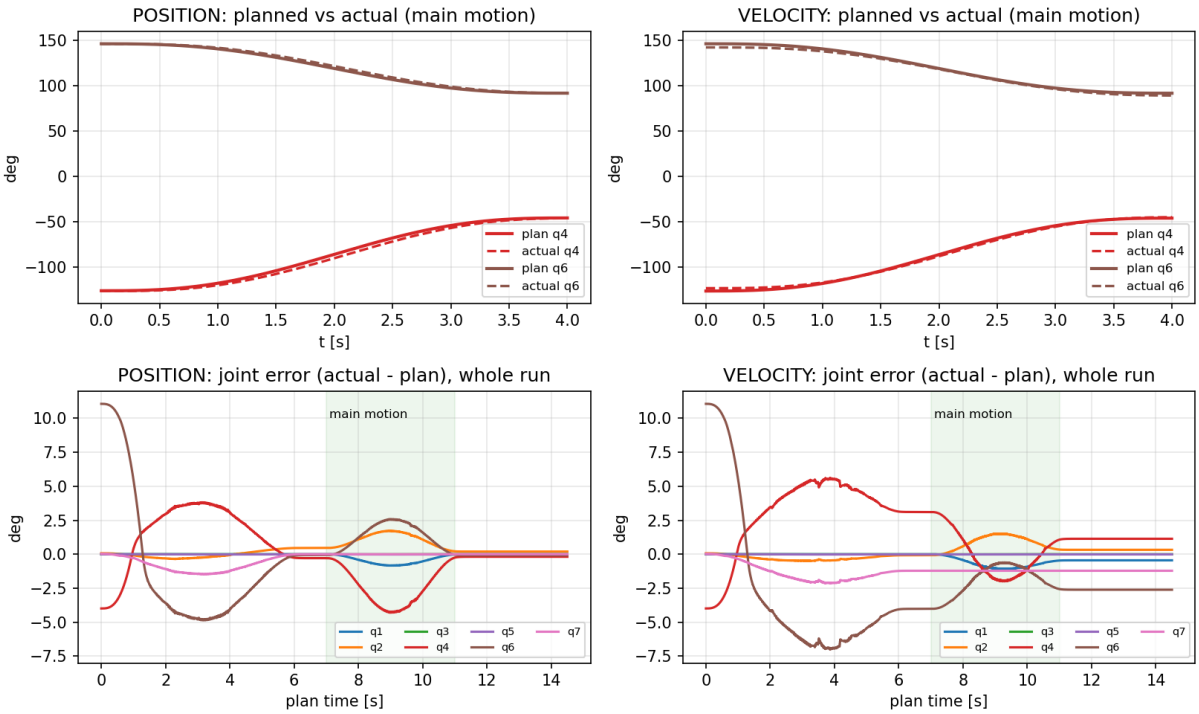


Figure 4: top: joints 4 and 6, plan (solid) vs actual (dashed); bottom: joint errors over the whole run.

Measure (main motion unless noted)	POSITION	VELOCITY
Hand position error, RMS / max	34.9 / 54.0 mm	23.7 / 38.7 mm
Largest joint error during the motion	4.3°	4.0°
Time lag behind the plan (joint 4)	115 ms	not meaningful
Settled joint error vs the goal	< 0.20°	up to 2.6°
Settled hand error vs the goal	4.3 mm	4.7 mm
Commanded motion delivered	99.8 %	97.7 %

5.3 Observations

- **Which mode tracks more closely?** POSITION. Its error is the servo lag only: about 115 ms (predicted k_v/k_p = 100 ms), proportional to joint speed, peaking mid-motion and returning to zero when the arm stops; the remaining 4.3 mm is gravity sag. In VELOCITY mode the hand RMS error looks smaller (23.7 vs 34.9 mm) only because the missing motion happens to cancel part of the lag; the joints stop up to 2.6° away from the goal.
- **Does VELOCITY mode drift or lag toward the end?** It drifts. The bridge integrates the messages it receives with no feedback, and only 97.7 % of the commanded motion arrived; every lost message is a permanent offset, so the error builds up during the motion and is the same after the arm has stopped. A lost message in POSITION mode is harmless because the next one carries the absolute target.
- **Why is motion lost?** The bridge keeps only the newest message (queue depth 1) and draws frames on the same thread as the 1 kHz physics. Repeating the VELOCITY run with other render rates: 5 Hz → 100 %, 20 Hz → 98 %, 30 Hz → 95 %, 60 Hz → 1 % of the motion delivered (one run each). The node itself published at 983–1000 Hz, so the loss is inside the bridge.

6. Conclusion and limitations

- The linear Jacobian (3 x 7) and its derivative follow directly from the DH table, agree with MuJoCo and with finite differences to machine precision / 1e-10, and are used to evaluate the singularity indicators along the path.

- The quintic joint-space plan starts and stops with zero speed and acceleration, respects position, speed (30 %) and acceleration (4 %) limits, and is protected by a singularity guard; the Team 6 path needs no back-off.
- POSITION mode follows the plan with a 0.1 s servo lag and no accumulated error. VELOCITY mode is open loop: with the template's 20 Hz it delivers 2 % of the motion, at 1 kHz about 97 % (less when the bridge draws more often), and from the simulator's start pose it cannot move joints 4 and 6 because the integrated targets leave the actuator range.
- Limitations: only the position Jacobian is monitored; the singularity thresholds are design choices; the results of each configuration come from one run; the start-up transient comes from the simulator's non-physical initial pose (joint 4 at 0, hand overlapping link 5).

How to run

```
colcon build --packages-select panda_kinematics && source install/setup.bash
ros2 launch mujoco_ros2 example.py            # bridge, control_mode POSITION or VELOCITY
ros2 run panda_kinematics trajectory_node     # the two mode lines in tick() must match the bridge
python3 src/panda_kinematics/tools/check_challenge2.py     # 22 self-checks (needs mujoco python)
```